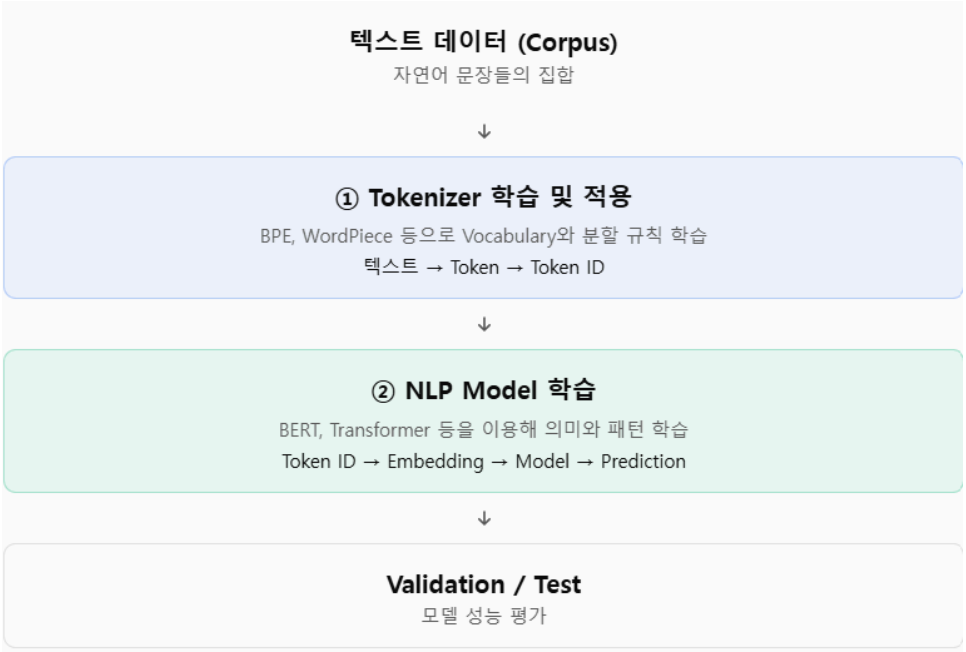
10주차 : 자연어 처리와 컴퓨터비전 심층학습 5~6장. ~TF-IDF

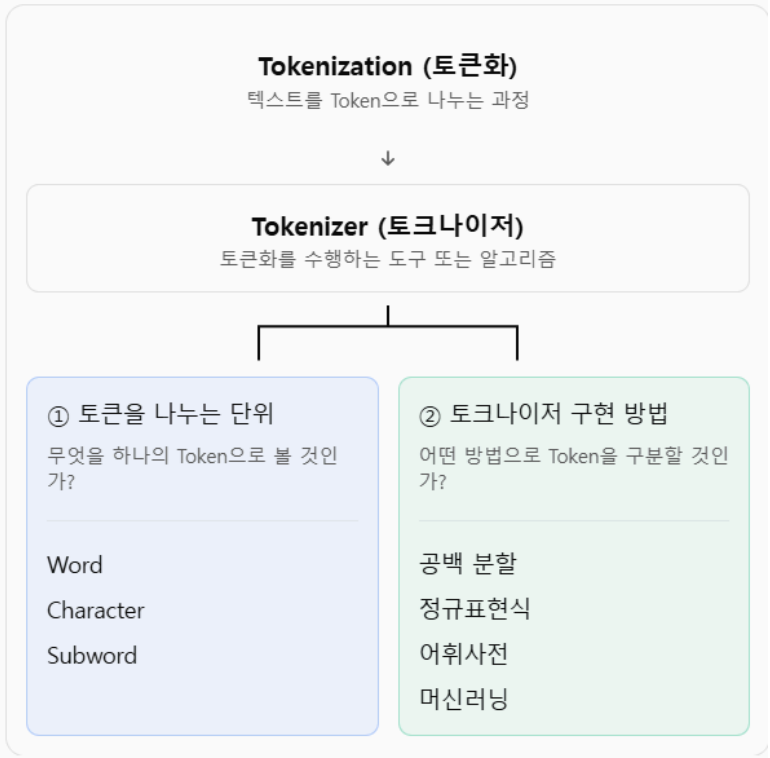
05. 토큰화

- 자연어(Natural Language) : 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어



- 자연어 처리 (Natural Language Processing, NLP) : 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술
    - 인간 언어의 구조, 의미, 맥락을 분석하고 이해할 수 있는 알고리즘을 위해 해결해야 할 문제
        a. 모호성 (ambiguity) : 사용되는 단어와 구가 맥락에 따라 여러 의미를 갖게 되어 모호한 경우가 많음
        b. 가변성 (variability) : 인간의 언어는 사투리, 강세, 신조어 등으로 인해 매우 가변적임
        c. 구조 (structure) : 인간의 언어는 문장이나 구의 의미를 이해할 때 구문(구조)을 파악하여 의미를 해석함

    ⇒ 위와 같은 문제를 이해하고 구분할 수 있는 모델을 만들려면 말뭉치(corpus, 대규모의 자연어)를 일정한 단위인 토큰(Token, 텍스트의 개별 단어, 구두점 또는 기타 의미 단위)으로 나눠야 한다.



- 토큰화(Tokenization) : 컴퓨터가 나연어를 이해할 수 있도록 말뭉치를 토큰으로 나눔
  - 토크나이저(Tokenizer) 사용 : 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 소프트웨어
  - 토크나이저를 구축하는 방법 [어떤 방법으로 Token을 구분할지]
    a. 공백 분할 : 텍스트를 공백 단위로 분리하여 개별 단어로 토큰화
    b. 정규표현식 적용 : 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할
    c. 어휘사전 적용 : 사전에 정의된 단어 집합을 토큰으로 사용
      - 어휘사전 (Vocabulary, Vocab)을 직접 구축하기 때문에 없는 단어나 토큰이 존재할 수 있음
        → 이와 같은 토큰을 OOV(Out of Vocab)이라고 부름
        ⇒ 어휘 사전 방법의 경우 OOV 문제를 고려하여 토큰화 하는 것이 중요
      - 큰 어휘사전을 구축할 경우 학습 비용이 증대되고, 차원의 저주에 빠질 수 있음
        → 차원의 저주 : 대부분의 값이 0이어서 벡터가 희소해짐 (Sparsity)
    d. 머신러닝 활용 : 데이터세트를 기반으로 토큰화하는 방법을 학습한 머신러닝 적용
- 토큰화 Unit : Word, Character, Morpheme, Subword [무엇을 하나의 Token으로 취급할지]

1. 단어 토큰화 (Word Tokenization) : 텍스트 데이터를 의미 있는 단위인 단어로 분리

- 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화 수행
- split 메서드를 이용 → 주어진 구분자를 통해 문자열을 리스트 데이터로 나눔
- 구분자를 입력하지 않을 경우 공백(Whitespace)을 기준으로 나눔

예제 5.1 단어 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거음은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = review.split()
print(tokenized)
```

출력 결과

```
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거음은', '최고!', '더불어', 'ost는', '더더욱',
'최고!!']
```

2. 글자 토큰화 (Character Tokenization) : 글자 단위로 문장을 나누는 방식
   - 비교적 작은 단어 사전을 구축할 수 있음 (어휘사전에 등록해야 하는 서로 다른 Token의 종류가 적다) =/= 문장을 나눴을 때 생기는 토큰의 총개수가 적다
     ⇒ 글자 토큰화는 문장을 더 많은 조각으로 나누지만, 그 조각들을 반복해서 재사용할 수 있기에 전체 어휘사전을 작게 유지할 수 있다.
   - 개별 토큰이 아무런 의미가 없음 → 자연어 모델이 각 토큰의 의미를 조합해 결과를 도출해야 함

예제 5.2 글자 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거음은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = list(review)
print(tokenized)
```

출력 결과

```
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', ' ', '시',
'각', '적', ' ', '즐', '거', '음', '은', ' ', '최', '고', '!', ' ', '더', '불', '어', ' ', 'o', 's',
't', '는', ' ', '더', '더', '욱', ' ', '최', '고', '!', '!']
```

3. 형태소 토큰화 (Morpheme Tokenization) : 텍스트를 형태소 단위로 나누는 토큰화 방법
   → 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류
   - 한국어와 같이 교착어(Agglutinative Language)인 언어에서 중요하게 수행됨
   - 형태소 : 실제로 의미를 가지고 있는 실제 단위 → 자립형태소 / 의존형태소

- 형태소 어휘 사전 : 형태소 정보를 포함하는 어휘 사전 → 단어가 어떤 형태소들의 조합으로 이루어져 있는지에 대한 정보를 담고 있음
- 대표 라이브러리
    - KoNLPy : 한국어 자연어 처리를 위해 개발된 라이브러리
      → 명사 추출, 형태소 분석, 품사 태깅 등의 기능을 제공함
    - NLTK(Natural Language ToolKit) : 주로 영어 자연어 처리를 위해 개발된 라이브러리
      → 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석과 같은 기능 제공
    - spaCy : Cython 기반으로 개발된 오픈 소스 라이브러리
      → 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반 자연어 처리 라이브러리

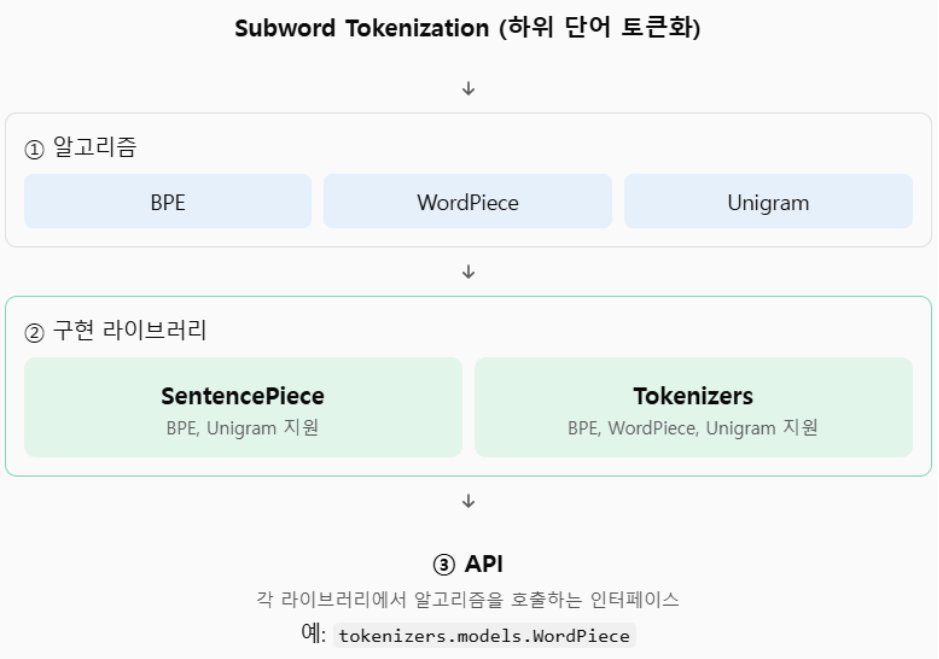
4. 하위 단어 토큰화 (Subword Tokenization) : 하나의 단어를 빈번하게 사용되는 하위 단어의 조합으로 나누어 토큰화 하는 방법
    - 현대 자연어 처리에서는 신조어 발생, 오탈자, 축약어 등을 고려해야 하기 때문에 분석할 단어의 양이 많아져 어려움을 겪음
    - 하위 단어 토큰화를 적용할 경우 단어의 길이를 줄여 처리속도가 빨라질 수 있음
    - OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제를 완화할 수 있음
    - 대표적인 하위 단어 토큰화 방법
        - 바이트 페어 인코딩 (Byte Pair Encoding, BPE) : 처음에는 텍스트를 작은 단위로 나누고, 자주 함께 등장하는 인접 Token 쌍을 반복적으로 합쳐 새로운 Token을 만드는 방식
        - 워드피스(Wordpiece) 알고리즘 : 학습 과정에서 확률적인 정보를 사용함
          → 모델이 새로운 하위 단어를 생성할 때 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택한다.

수식 5.1 글자 쌍 병합 점수 수식

$$score = \frac{f(x,y)}{f(x),f(y)}$$

f는 빈도(frequency)를 나타내는 함수이며, x와 y는 병합하려는 하위 단어를 의미한다. 그러므로 $f(x, y)$는 x와 y가 조합된 글자 쌍의 빈도를 의미한다. 즉, xy 글자 쌍의 빈도가 된다. 그러므로 $score$는 x와 y를 병합하는 것이 적절한지를 판단하기 위한 점수가 된다.

        - 유니그램 모델



- 센텐스피스 (Sentencepiece) 라이브러리
- 토크나이저스 (Tokenizers) 라이브러리 : 정규화 (Normalization)와 사전 토큰화 (Pre-tokenization)을 제공함
  - 정규화 : 일관된 형식으로 텍스트를 표준화, 모호한 경우를 방지하기 위해 일부 문자를 제거하거나 대체
  - 사전 토큰화 : 입력 문장을 토큰화하기 전에 단어와 같은 단위로 나누는 기능 → 이후 토큰화 알고리즘이 처리할 입력 단위를 준비하는 단계
- 코포라 (Korpora) 라이브러리 : 한국어 말뭉치(Corpus)를 쉽게 가져와 사용할 수 있게 해주는 python 라이브러리

06. 임베딩

- 컴퓨터는 텍스트 자체를 이해할 수 없음 → 텍스트를 숫자로 변환하는 텍스트 벡터화 과정이 필요함
- 텍스트 벡터화 (Text Vectorization) : 텍스트를 숫자로 변환하는 과정
  - 원-핫 인코딩 : 각 단어를 고유한 색인 값으로 매핑한 후, 해당 색인 위치를 1로 표시하고 나머지 위치는 모두 0으로 표시하는 방식
  - 빈도 벡터화 (Count Vectorization) : 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식

→ 벡터의 희소성 (sparsity)가 크다는 단점이 있고, 두 문장이 의미적으로 유사하다고 해도 벡터가 유사하게 나타나지 않을 수 있다는 단점이 있음

- 워드 임베딩 기법 (Word Embedding) : Word2Vec이나 fastText 등과 같이 단어의 의미를 학습해 표현

  → 고정된 임베딩을 학습하므로 다의어나 문맥 정보를 다루기 어렵다는 단점이 있음

- 동적 임베딩 기법 (Dynamic Embedding) : 인공 신경망 활용

- 언어 모델 (Language Model) : 입력한 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델
  → 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측수행
  - 주어진 문장 뒤에 나올 수 있는 문장은 매우 다양함
    → 문장 단위로 확률을 계산하는 것은 어려운 일임
    ⇒ 문장 전체 대신 하나의 토큰 단위로 예측하는 방법인 자기회귀 언어 모델이 고안됨
- 자기회귀 언어 모델 (Autoregressive Language Model) : 입력된 문장들의 조건부 확률(이전 단어들의 시퀀스가 주어짐 → 다음 단어의 확률을 계산)을 이용해 다음에 올 단어를 예측

수식 6.1 언어 모델의 조건부 확률

$$P(w_t \mid w_1, w_2, \cdots, w_{t-1}) = \frac{P(w_1, w_2, \cdots, w_t)}{P(w_1, w_2, \cdots, w_{t-1})}$$

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_t \mid w_1, w_2, \cdots, w_{t-1}) = P(w_1)P(w_2 \mid w_1)\cdots P(w_t \mid w_1, w_2, \cdots, w_{t-1})$$

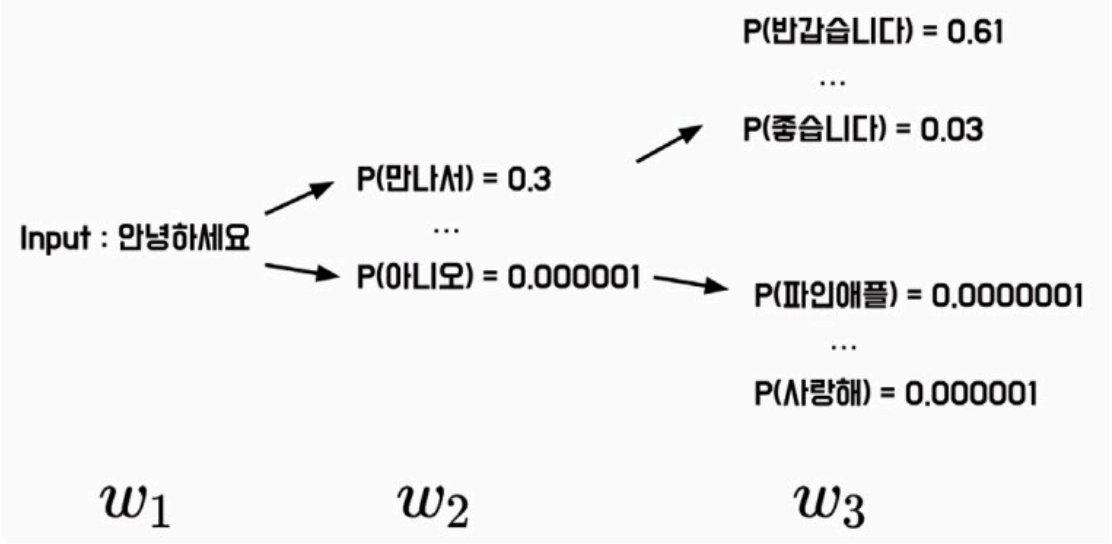


그림 6.2 조건부 확률의 연쇄법칙을 통한 자기회귀 언어 모델 구조

- 입력토큰 w1 을 바탕으로 다음 토큰 w2가 등장할 확률 계산
- 이후 출력값인 w2가 다시 입력값이 되어 w3이 등장할 확률 계산

  ⇒ 모델의 출력값이 모델의 입력값으로 사용되는 특징 때문에 자기회귀라는 이름이 붙음

- 통계적 언어 모델 (Statistical Language Model) : 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석함 → 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률을 예측함
  - 마르코프 체인 (Markov Chain) : 빈도 기반의 조건부 확률 모델 중 하나
    → 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측함

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서} \mid \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} \mid \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

  - 단어의 순서와 빈도에만 기초해 문장의 확률을 예측함 → 관측한 적이 없는 데이터를 예측하지 못하는 문제가 발생할 수 있음 (= 데이터 희소성 문제, Data Sparsity)

- N-gram : (가장 기초적인 통계적 언어 모델) 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정함
  - 연속된 N개의 단어를 하나의 단위로 취급하여 추론하는 모델
  - N=1 : 유니그램, N=2 : 바이그램, N=3 : 트라이그램



그림 6.3 N이 1, 2, 3인 N-gram

  - N-1개의 토큰만을 고려해 확률을 계산함

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \cdots, w_{t-N+1})$$

w_t는 예측하려는 단어, w_{t-1}부터 w_{t-N+1}까지는 예측에 사용되는 이전 단어들을 의미한다. 이전 단어들의 개수를 결정하는 N의 값을 조정하여 N-gram 모델의 성능을 조절할 수 있다.

- TF-IDF (Term Frequency-Inverse Document Frequency) : 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법 → 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적인 가중치를 의미함
  ⇒ Bag-Of-Words, BoW 벡터의 각 단어에 해당하는 값에 가중치를 부여하는 방법
  - BoW : 텍스트를 단어의 등장 여부나 빈도에 따라 벡터로 표현하는 방식
  - 단어 빈도 (Term Frequency, TF) : 문서 내에서 특정 단어의 빈도수를 나타내는 값

수식 6.5 단어 빈도

$$TF(t,d) = count(t,d)$$

  - 문서 빈도 (Document Frequency, DF) : 한 단어가 얼마나 많은 문서에 나타내는지를 의미함

수식 6.6 문서 빈도

$$DF(t,D) = count(t \in d : d \in D)$$

- 역문서 빈도 (Inverse Document Frequency, IDF) : 전체 문서 수를 문서 빈도로 나눈 다음에 로그를 취한 값 → 문서 내에서 특정 단어가 얼마나 중요한지를 나타냄

수식 6.7 역문서 빈도

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

⇒ TF-IDF는 문서 빈도와 역문서 빈도를 곱한 값으로 사용함

수식 6.8 TF–IDF

$$TF - IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

- 문서 내에 단어가 자주 등장하지만 전체 문서 내에 해당 단어가 적게 등장 → TF-IDF 값 커짐
- 관사, 관용어 등의 가중치는 낮아짐